Bor

Policy Management for Linux Workstations with Bor

Open source, self-hosted, real-time.

Blagovest Petrov · Founder, **Vute.Tech**

OpenFest 2026 · 7 November · Interpred WTC, Sofia · hall "Varna"

Policy management for Linux desktops

Who am I?

Blagovest Petrov

Founder of **Vute.Tech** · DevOps / Platform Engineering

- Many years of Linux in corporate environments: servers, infrastructure, automation
- Long-time free software user and OpenFest regular

blago.petrov@vute.tech · vute.tech · getbor.dev

Quick poll (hands up)

- Who manages Linux **workstations**, not just servers?
- More than 10? More than 100?
- With Ansible, Puppet or Salt?
- With shell scripts and cron?
- By hand, over SSH, when somebody complains?
- *Nobody? Not even your grandma's laptop?*

> Remember which hand you raised. We will come back to it before the demo.

CHAPTER 01

The problem

2027 is the year of the Linux desktop. *(Yes, really?)*
But are we ready to welcome it?

Why now

14.10.2025

Windows 10 reached end of support.

~240M PCs that cannot run Windows 11
Canalys estimate, Dec 2023: working hardware, expired software

28% of Windows desktops still on Windows 10, one year later
StatCounter, September 2026

~6% Linux desktop share in Bulgaria, above the world's 4.5%
StatCounter, September 2026; small sample, noisy

Public sector moving to Linux desktops

- **Schleswig-Holstein**: "Linux+1" programme for ~30 000 workstations; the State Chancellery runs on Linux since September 2026
- **Lyon**: 4 000+ workstations from Windows and Office to Linux and OnlyOffice
- **France, DINUM**: its own workstations to Linux; every ministry files a roadmap off non-European dependencies by autumn 2026
- **Netherlands**: a NixOS-based government workplace, eight municipalities piloting in 2026
- **Gendarmerie Nationale**: ~70 000 Linux desktops, quietly, since 2008

> The motive is no longer "it's free". The motive is **control**: of data, of cost, of vendor dependency. Control needs management.

The awkward question

> **"Great, we're moving to Linux!**
> **… so how do we manage all these machines?"**
>
> every IT director, about a week after the decision

- This question keeps organisations on Windows **more** than application compatibility does.
- The usual answer: "we'll figure something out". A script, a cron job, and one person who knows how it works.
- Without a convincing answer, the migration stops right there, in the meeting room.

Linux has the mechanisms. It lacks the management plane.

- Every major desktop application has an **official, admin-locked** configuration mechanism
- Each one is a different file format in a different place
- Nothing distributes them, watches them, or reports whether they are actually in effect
- Windows admins have had one model for this since 2000: Group Policy

```
/etc/firefox/policies/policies.json
/etc/opt/chrome/policies/managed/*.json
/etc/xdg/kdeglobals            [KDE Action Restrictions]
/etc/dconf/db/local.d/*  +  locks/
/etc/polkit-1/rules.d/*.rules
/etc/firewalld/zones/*.xml
/etc/security/time.conf         (pam_time)
/etc/apt/sources.list.d/*.sources   /etc/yum.repos.d/
/etc/systemd/system/*.d/*.conf
/etc/selinux/   /etc/apparmor.d/
/etc/crypttab  +  clevis / systemd-cryptenroll
/var/lib/flatpak/repo/config  +  flatpak remotes
```

> All documented. None of it centralised. Twelve formats, twelve places, zero feedback.

CHAPTER 02

Bor

What it is and how it works.

What is Bor?

Policy management for Linux desktops. Think Group Policy, built Linux-first.

CONSOLE
Central web console
Policies, node groups, bindings, RBAC, audit log. PatternFly 6, WCAG 2.2 AA, dark mode.

AGENT
Lightweight agent
Go, one static binary, a systemd service. Listens on no port; only dials out.

DELIVERY
Real-time
Persistent gRPC stream over mTLS. A released policy lands on every node in seconds.

ENFORCEMENT
Continuous
Writes the native config, watches it, restores it if touched, reports compliance.

LINUX-FIRST | SELF-HOSTED | OPEN SOURCE · AGPL-3.0 | AIR-GAPPED CAPABLE

"For me, Bor is the one that comes closest to Windows GPOs, and the one with the most potential." Karl Certa, syspirit.fr, 6 August 2026

What Bor is *not*

- **A directory.** It does not replace FreeIPA, Samba AD or LDAP. It integrates with them: sign-in, groups, Kerberos.
- **Configuration management.** It is not Ansible, Puppet or Salt. They prepare the machine; Bor keeps it in line every day after that.
- **A provisioning tool.** It does not install operating systems or manage their lifecycle.
- **A scripting engine.** A policy is a typed, declarative document with a schema, never code. There is no "run this on every machine".

> Bor is the policy layer that sits *beside* those tools: Ansible installs the agent, Bor takes it from there.

Why "Bor"?

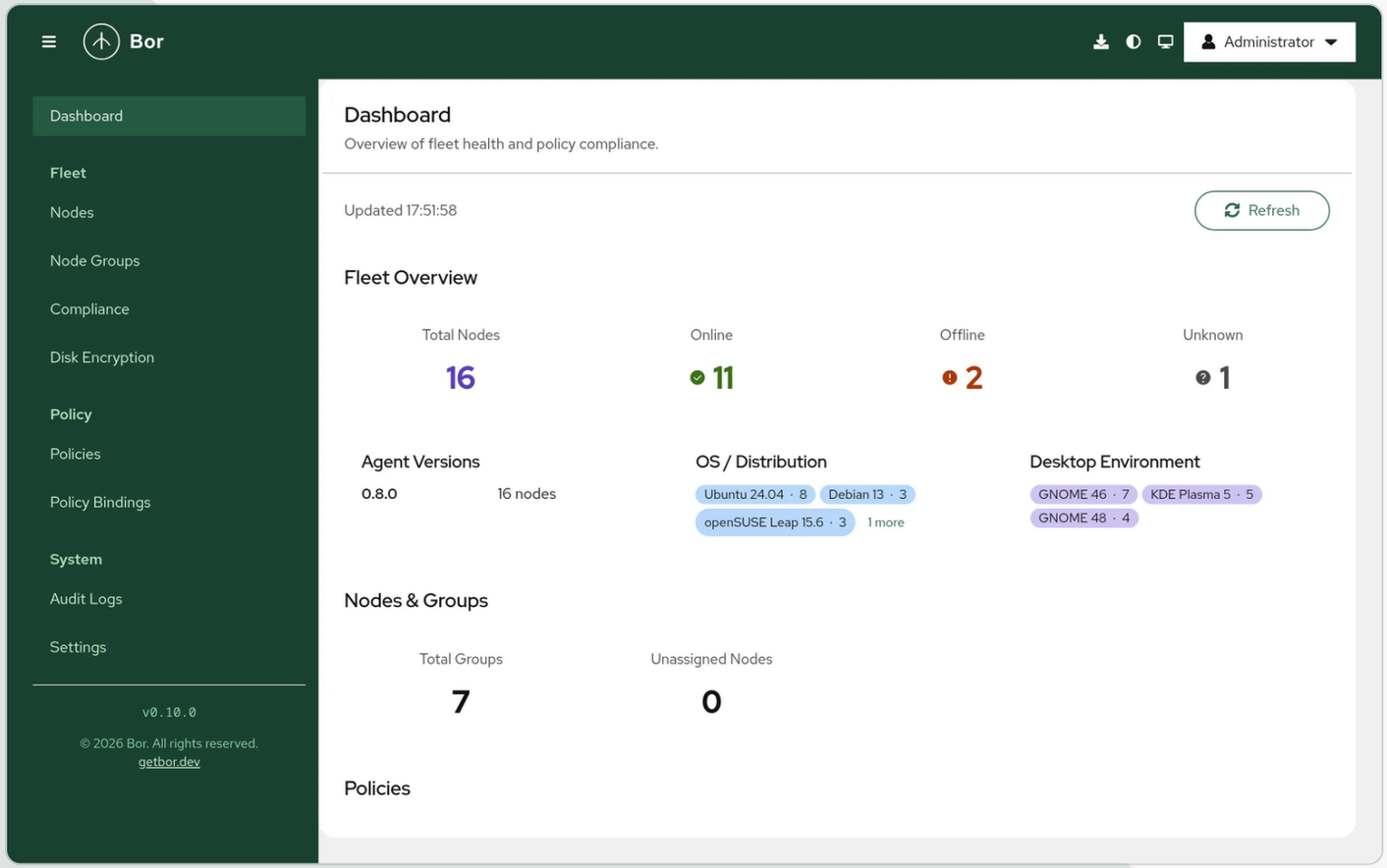


Бор. „Прекрасно българско трибуквие.“ The Slavic word for *pine*.

- Evergreen.
- Grows where little else does: on rock, on sand, in the wind.
- Needs little. Lives long.
- Has branches.
- A Slavic word anyone in the world can pronounce.

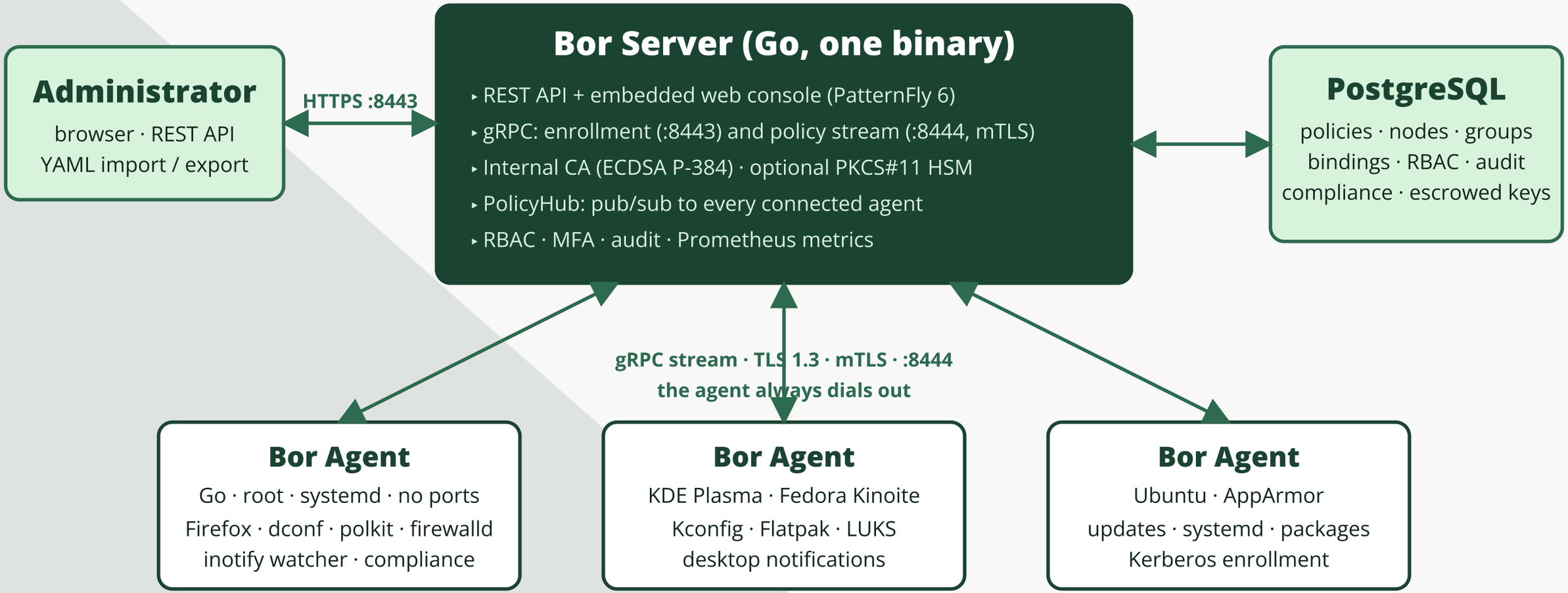
Search for `getbor.dev`. Searching for "бор" alone gets you lumber prices.

The console at a glance



- **Fleet**: nodes, node groups, compliance, disk encryption
- **Policy**: policies, bindings, import and export
- **System**: audit logs, users, groups, roles, settings
- Dashboard: online / offline / unknown, agent versions, distributions, desktop environments, policies by state and type
- Server-side pagination, filtering and sorting; deep links for every page
- Light, dark and high-contrast themes

Architecture



Enrolling a machine

1. An admin generates a **one-time token** in the console: 256-bit, 5-minute TTL, single use, only its hash is stored. *Or* the machine is domain-joined and has a Kerberos ticket.
2. `bor-agent --token ...`: the agent generates an **ECDSA P-256** key locally and a CSR.
3. Enroll RPC over TLS: token (or Kerberos) + CSR.
4. The server validates, signs with the internal CA and creates the node in the token's group.
5. Certificate and CA are stored in `/var/lib/bor/agent`. From here on: **mTLS**.

The private key **never leaves the machine**.

The token is bound to a node group: policies start flowing immediately.

Certificates live 90 days and **renew themselves**. Revocation is checked on every RPC.

Kerberos: no token at all. The machine keytab proves identity (AD, Samba AD, FreeIPA).

Policies arrive in seconds (not at the next cron run)

Agent		Server
connects with mTLS and states its last known revision	`-->`	`SubscribePolicyUpdates(rev)`
receives a **SNAPSHOT** (new agent, or revision too old)	`<--`	all current policies + end-of-snapshot marker
applies atomically, reports **compliance**	`-->`	stored per node and per policy
… the stream stays open …		
		an admin releases a policy or changes a binding
receives only the **delta**: CREATED / UPDATED / DELETED	`<--`	PolicyHub broadcasts to every connected agent
applies, notifies the user, reports again	`-->`	dashboard updates

PolicyHub

- in-memory pub/sub, monotonic revision
- 1000-event ring buffer for deltas on reconnect; revision persisted in PostgreSQL

Offline?

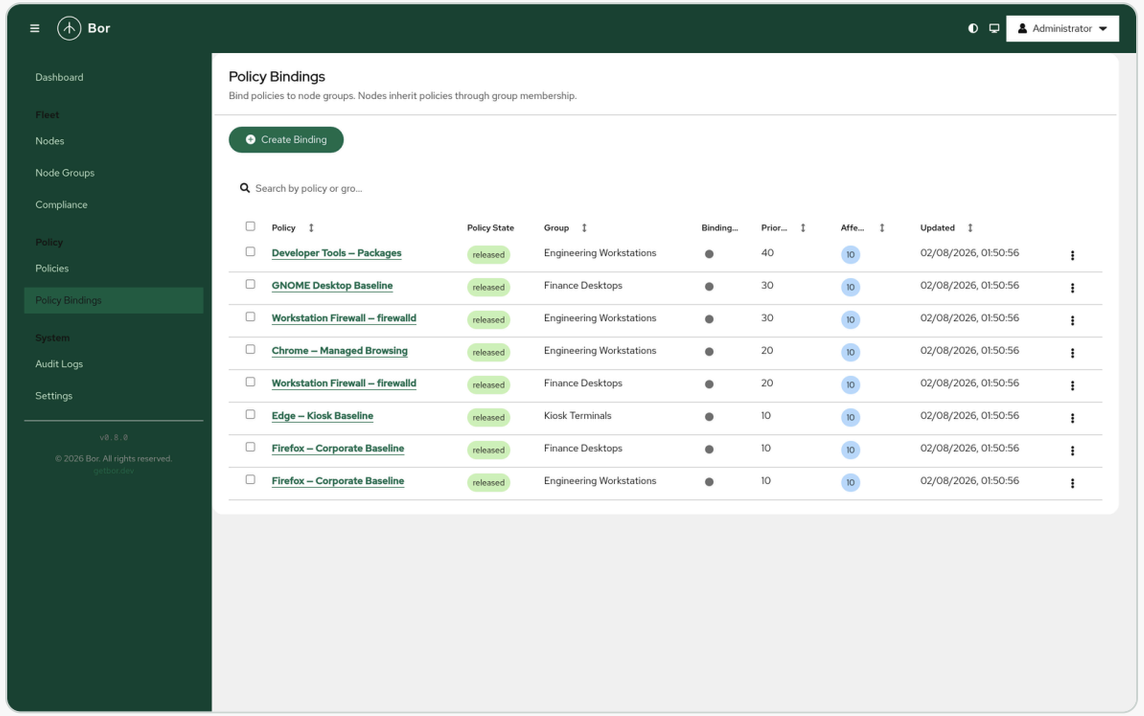
- the last config is a native file: Firefox reads it with or without the agent
- reconnect with backoff, capped at 60 s

Conflicts

- every binding has a **priority**; for the same key the higher one wins (Polkit is the exception: lower number, first match)

Node groups, bindings and precedence

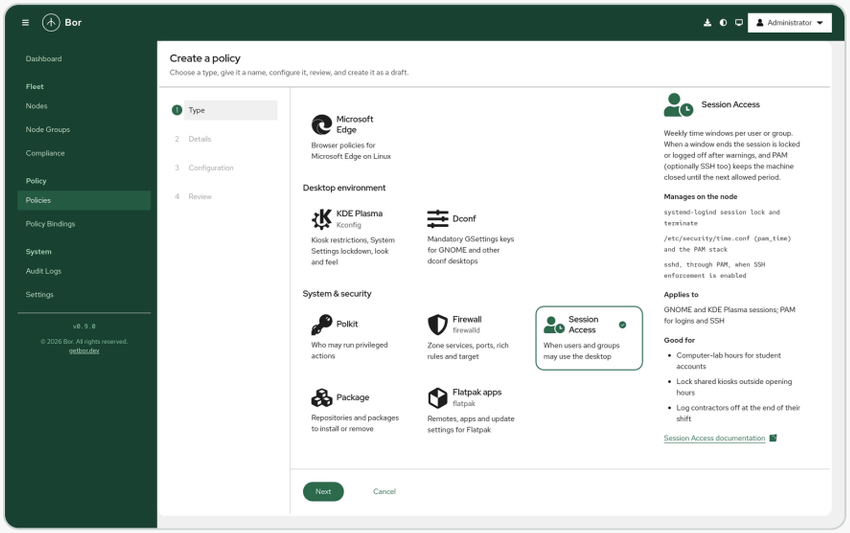
- **Node groups** are many-to-many: a laptop can be in *Finance Desktops* and *Developers* at once
- **Bindings** link a released policy to a group, with an enabled flag and a **priority**
- Same-type policies merge by priority: the higher number wins per key (Polkit: the lower one, its rules file is read first)
- Example: *USB storage block* at 50 for everyone, *Speaker USB exception* at 10 for one group
- Membership changes reach connected agents at once; enrollment tokens are issued per group
- Bindings carry an "affected nodes" count so you see the blast radius before you save



Policy lifecycle and editors

Draft > **Released** > **Archived** only released policies reach agents; released content is read-only, edit a new draft

- **Create wizard**: type, details, configuration, review. Each type card explains what it manages on the node and what it is good for
- **Typed editors** per policy type: tree editors with setting previews, day-and-time builders, catalog search, no raw JSON unless you want it
- **Proto-driven catalogues**: Firefox, Thunderbird, Chrome and Edge settings are generated from protobuf annotations, one source of truth for server, agent and UI
- **Safety rails**: unsaved-changes guard, JSON validation for Chrome and Edge values, confirmation for destructive type changes, type-to-confirm deletes
- **Import preview**: see exactly what a YAML bundle will create before anything is written



16 policy types

BROWSERS & MAIL

- **Firefox**
  ESR and Flatpak; policies.json
- **Thunderbird**
  updates, privacy, security
- **Chrome / Chromium**
  managed policies JSON
- **Microsoft Edge**
  Edge for Business on Linux

DESKTOP ENVIRONMENT

- **KDE Plasma**
  Kconfig / Kiosk: restrictions, lockdown
- **Dconf**
  GNOME and any GSettings desktop, locks
- **Session Access**
  when users and groups may use the desktop
- **Flatpak**
  remotes, apps, update settings, catalog

SYSTEM & SECURITY

- **Polkit**
  who may run privileged actions
- **Firewall**
  firewalld zones, services, rich rules
- **Package**
  repositories and packages: apt, dnf, zypper
- **Disk encryption**
  V0.10
  LUKS2, TPM2, Tang, recovery keys
- **Services & timers**
  V0.10
  systemd units, timers, scheduled tasks
- **SELinux**
  V0.10
  mode, booleans, exceptions, modules
- **AppArmor**
  V0.10
  profile modes, user namespaces
- **Software updates**
  V0.10
  unattended updates, schedule, the reboot

Every type uses the application's **official** system-wide mechanism: no patched binaries, no LD_PRELOAD. The configuration keeps working even if the agent is stopped.
Product marks belong to their respective owners.

Browsers, mail and desktop environments



Browsers and mail

Firefox, Thunderbird, Chrome / Chromium, Microsoft Edge

- Homepage, search, extensions (pinned, blocked, force-installed), DNS over HTTPS, privacy settings, password manager, update channel, certificates, proxies, popups
- Firefox and Chromium **Flatpak** installs are covered too
- Settings catalogue generated from the protobuf schema, with previews and validation
- The same editor model across all four: one baseline, every browser



KDE Plasma (Kconfig / Kiosk)

- Overlay under `/etc/xdg`, immutable keys, KDE Action Restrictions, System Settings module lockdown, look and feel, settings scoped per Unix group



Dconf (GNOME and friends)

- System database plus **locks** for any GSettings schema; a built-in key catalogue seeded on the server
- Screen lock, idle, user switching, wallpaper, default apps, privacy settings

Works on X11 and Wayland alike: it is files, dconf and logind, not the display server.

System and software

POLKIT

Who may do what

Rules in `rules.d` with user, group and **variable conditions**. The classic: non-IT users cannot mount removable drives; developers get it back via a higher-priority binding.

FIREWALLD

Zones as policy

Services, ports, rich rules and target per zone. Default-deny inbound, SSH only from the management subnet. The policy can start and enable firewalld if it is not running.

PACKAGE

Repositories and packages

APT deb822, DNF/YUM and Zypper repos with GPG keys, written in the node's native format. Packages *present / absent / latest*, version pins, optional flag. Installs via PackageKit. **One-click import** of Ubuntu PPAs, Fedora COPR and openSUSE .ymp.

FLATPAK

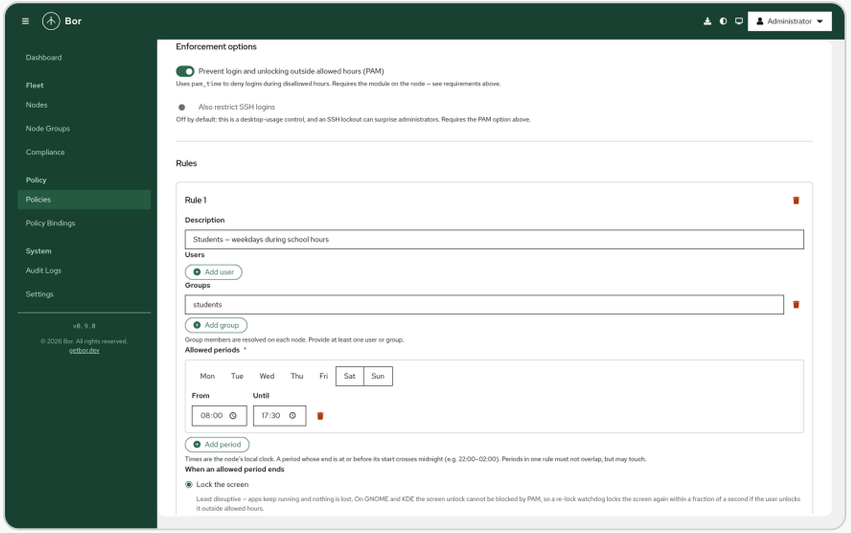
Remotes, apps and a catalogue

Remotes with GPG key, subset (Flathub *verified*), allow and deny filters, priority. Apps *present / latest / absent* per remote and branch, periodic updates, runtime cleanup. Flathub indexed from AppStream; air-gapped servers upload the index by hand. Existing `flathub` remotes are adopted, not duplicated.

Session Access (born from a Hacker News comment)

"Could Bor enforce screen time on laptops given to children, better than iOS and Android do?" Three weeks later it shipped in v0.9.0.

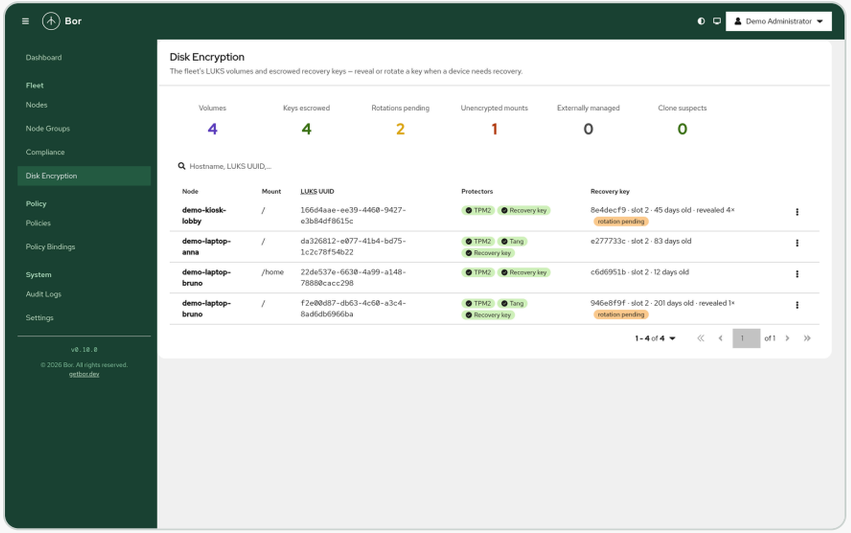
- **Weekly schedules** per user or group, in node-local time, windows that cross midnight
- **Layer 1, logind** (always on): warnings at 30, 15 and 5 minutes (the last one bypasses do-not-disturb), then **lock** or **log off**; a re-lock watchdog keeps the session closed
- **Layer 2, pam_time** (optional): blocks new logins outside the window, optionally for SSH too; keeps enforcing while the agent is down
- Distro-aware PAM: `pam-auth-update` on Debian and Ubuntu; a documented one-time step on authselect systems
- Guard rails: root is never restricted, atomic writes with self-check, fail-open removal on unbind



Computer-lab hours, shared kiosks outside opening hours, contractors at end of shift.

Disk encryption (what AD does for BitLocker, for Linux)

- **Verifies** that the volumes that should be LUKS2-encrypted actually are; unencrypted mounts are non-compliant
- **Protectors, not passphrases**: TPM 2.0 with explicit PCR profiles (Secure Boot by default), **Tang** servers with pinned thumbprints and a threshold, or both
- **Recovery key per volume**, escrowed envelope-encrypted on the server; reveal needs a dedicated permission, every reveal is audited, keys rotate after use
- **Auto-reseal**: a firmware update changes TPM measurements; the next boot falls back to Tang or the key, then the agent re-enrolls the TPM
- Adoption paths: existing Clevis/Tang setups, a one-time bootstrap key from Kickstart or autoinstall, or `bor-agent luks adopt`
- Never encrypts a running disk in place; detects TPMs owned by other FDE stacks and reports them as externally managed



Fleet view: volumes, keys escrowed, rotations pending, unencrypted mounts, externally managed, clone suspects.

Software updates (Bor owns the reboot)

- Drives the distribution's own engine: `dnf-automatic`, `unattended-upgrades`, Zypper, and `transactional-update` on MicroOS, Aeon, Kalpa and SLE Micro
- **Mode and schedule**: security-only or everything, nightly window, offline installs at reboot on Fedora
- **The reboot**: only inside the maintenance window, only when nobody is logged in, forced after N days with a warning to the user
- **Holds** for packages that must not move, every hold change audited
- **Firmware**: nodes with fwupd list pending firmware updates
- **CVE export** of pending updates from the node list or a node page; Updates column and filter in the fleet view
- Node actions: *Check now* and *Apply now* from the console

Patch management story: security updates every night, the reboot an update needs happens between 03:00 and 05:00 when nobody is logged in, forced after a week with a warning.

Compliance items for the engine, the mode, the schedule, pending updates and the reboot state, each with its CIS control number where one applies.

Services, timers and mandatory access control (new in v0.10)

SERVICES & TIMERS



systemd

- Unit state: enabled, disabled, masked; unit names come from the inventory agents report
- Timer schedules and **scheduled tasks**: a script or program deployed as a Bor-owned unit, under a SELinux or AppArmor profile if you want
- Optional preset file; requires the separate `systemd:exec` permission

SELINUX



Fedora, RHEL, derivatives

- Enforcing mode, booleans, per-domain **permissive exceptions** (the misbehaving VPN client, not the whole system), custom modules
- Editor shows the node's or the fleet's current booleans and domains; `setenforce 0` is reverted

APPARMOR



Ubuntu, Debian, openSUSE

- Profile modes (enforce, complain, disable), the **user namespace restriction**, custom profiles such as the four-line stub that lets an in-house Electron app sandbox
- Agents collect denials; the node page proposes AppArmor rules and SELinux modules from them

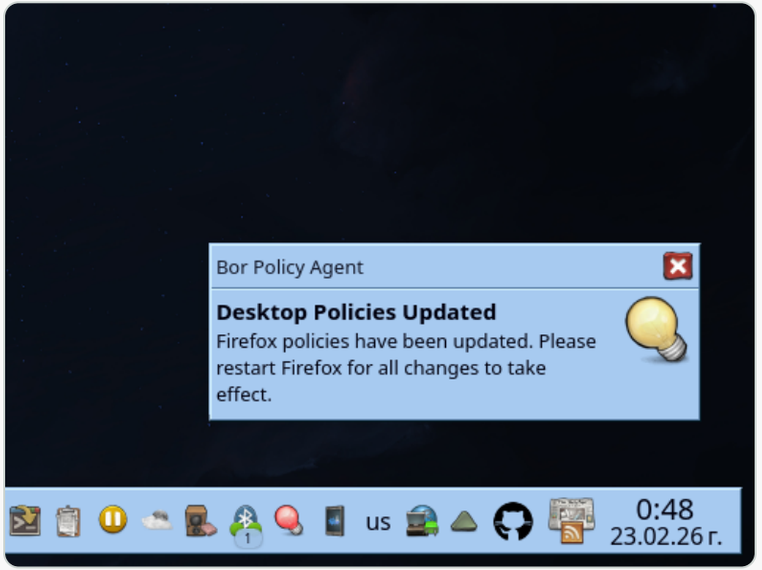
Compliance items for these types carry **CIS benchmark control numbers**, and policies need the `mac:author` / `mac:operate` permissions.

When somebody "fixes" the file by hand

1. A user with sudo opens `policies.json` and changes the homepage
2. **inotify** wakes the agent the moment the file is written
3. The file is **restored** atomically (temp + rename) before Firefox reads it
4. **Tamper event**: path, plus PID, command and user of the process holding the file
5. Audit log on the server, desktop notification, node status updated

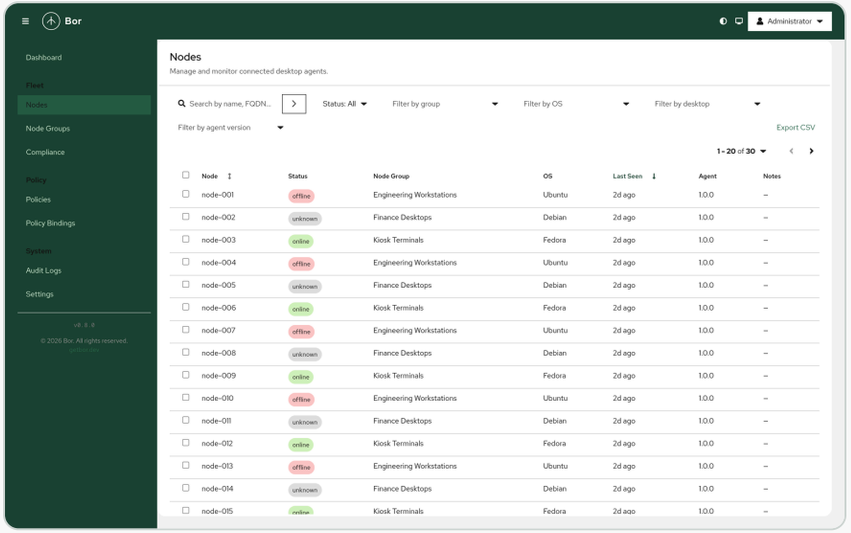
- Every file-based type is watched: browser policies, Kconfig, dconf, polkit, repos, firewalld zones, `time.conf`, Flatpak remotes, systemd drop-ins and tasks
- systemd, SELinux and AppArmor drift is caught through D-Bus signals and audit records and re-asserted the same way

> Honest part: local root can **always** stop the agent. That is true on every OS. The goal is visibility and accountability, not DRM, and the server sees when an agent stops checking in.



Compliance as data, node by node

- Every policy item reports **COMPLIANT** **NON-COMPLIANT** **INAPPLICABLE** **ERROR** **UNKNOWN**
- *Inapplicable* is honest: a node without Flatpak is not "broken"
- Compliance page: per node, per group and per policy, with server-side filters and sorting
- **Node page**: distribution, desktop environment, agent version, heartbeat; sections for mandatory access control, software updates and firmware, LUKS volumes
- Items carry CIS control numbers where they apply
- Fleet views: Updates column and filter, disk-encryption summary, CVE export



> **Bor Enterprise** adds point-in-time **reports**: snapshots with hashed evidence, daily trends, scheduled delivery, signed PDF/A documents, waivers and sign-off, ISO/IEC 27001 Annex A mapping.

Audit and observability

Audit log

- Every event with actor, IP, timestamp, action and target: sign-ins and MFA, permission changes, policy edits and releases, enrollments, tamper events, holds, and **denied** requests
- Export as CSV or JSON, hardened against spreadsheet formula injection
- Syslog export to your SIEM as **CEF** or **OCSF 1.3** JSON
- GDPR controls: IP anonymisation, retention period, self-service data export, nothing sensitive is ever logged

Metrics and users

- Prometheus `/metrics` on its own port, loopback by default, optional bearer token and TLS
- Grafana dashboard and alerting examples shipped with the docs
- Agents report heartbeats; the dashboard shows online, offline and unknown nodes
- Users get a **desktop notification** when policies change and when a session is about to lock

> Every authentication event, privilege change and policy modification is recorded with timestamp, actor, action and target, and can be exported in a structured format.

Identity and access control

Who can sign in

- Local users (PBKDF2-SHA256, 600 000 iterations), self-service password change
- **LDAP**: Active Directory, FreeIPA, OpenLDAP; group mapping
- **MFA**: TOTP with backup codes, **WebAuthn / FIDO2** keys; enforceable for everyone, enrollment can be forced at first login
- Agents authenticate with certificates, or **Kerberos** on domain-joined machines

What they may do

- Built-in roles (Super Admin, Org Admin, Policy Editor, …) and custom roles with **per-action permissions**: `policy:release`, `systemd:exec`, `mac:author`, `report:view`, …
- Roles on users and on user groups; privilege-escalation guards: you cannot grant what you do not hold, not even through a group
- Server-side guard against deleting, disabling or demoting the last Super Admin
- Type-to-confirm on every destructive action; denied changes are audited
- **Enterprise**: SSO with OIDC and SAML 2.0, SCIM 2.0 provisioning, roles scoped to node groups (delegated administration)

Security by default

PKI and transport

Role	Algorithm	Lifetime
Internal CA	ECDSA P-384	10 years
Server certificate	ECDSA P-256, or ACME / Let's Encrypt	365 days
Agent client certificate	ECDSA P-256	90 days, auto-renewed

- TLS 1.3 and mandatory mTLS on the agent port; AEAD cipher suites only
- CA generated on first start; a **PKCS#11** build keeps its key in an HSM
- Revocation checked on every RPC against the database, no stale CRLs
- Crypto from a **FIPS 140-3** validated module (Go, CAVP A6650); BSI TR-02102 and SOG-IS algorithm choices

Hardening

- Typed protobuf on the wire; paths and names sanitised; atomic file writes
- SSRF-hardened import helpers with DNS pinning; downloads served from an `os.Root`-scoped tree
- Rate-limited auth routes, CSRF tokens, size and keepalive limits on gRPC
- Initial admin password written to a root-only file, never to the log

> No third-party audit yet. The code, the crypto and the list of known limitations are public: check us.

Install and operate

Server

- One binary + PostgreSQL. No queues, no sidecars.
- `podman-compose up -d`, deb/rpm with a systemd unit, container image (UBI, non-root, read-only FS), Helm chart
- TLS: built-in CA, your own certificate, or **ACME**
- Migrations run on start; backup = PostgreSQL + `/var/lib/bor/pki`; restart-safe state

Agent

- Packages: **deb, rpm, apk, Arch**; x86_64, aarch64, ppc64le
- Debian, Ubuntu, Fedora, RHEL, openSUSE, Alpine, Arch; immutable: Kinoite, Silverblue, MicroOS
- The server hosts **signed repositories** for the agent: nodes need no internet
- **Deploy agent wizard**: one generated script trusts the CA, adds the repo, installs, enrolls and starts

```
# /etc/bor/config.yaml on the node
server:
  address: "bor.example.org:8443"
# then:  bor-agent --token <ONE-TIME TOKEN>  &&  systemctl enable --now bor-agent
```

Policies as code

```yaml
apiVersion: bor.dev/v1
kind: Policy
metadata:
  name: firefox-corporate-baseline
  displayName: Firefox - Corporate Baseline
spec:
  type: Firefox
  content:
    DisablePocket: true
    Homepage:
      Locked: true
      StartPage: homepage
      URL: https://intranet.example.com
---
apiVersion: bor.dev/v1
kind: PolicyBinding
metadata:
  name: firefox-corporate-baseline-finance
spec:
  policy: firefox-corporate-baseline
  group: Finance Desktops
  priority: 10
```

- Export and import **YAML or JSON** bundles of policies and bindings
- The same protobuf model for server, agent, console and file: no "almost the same" schema
- Strict validation at the import boundary, **preview before writing**, skip-existing mode
- Everything the console does goes through the REST API
- Staging to production, backups, shared baselines, evidence for auditors
  - Git sync and signed bundles: on the roadmap

CHAPTER 03

The project

Licence, what comes next, and a demo.

Editions and licence

- From v0.10.0: **AGPL-3.0** (previously LGPL). Improvements come back even when Bor is offered as a service.
- **CLA** modelled on Apache's: you keep your copyright; it is written in that your contribution stays open source.
- The Grafana and Mattermost model: the community edition for everyone, **Bor Enterprise** for large organisations funds the development of both.

> **Promises:** the community edition stays free forever, no node limits, no keys · what is open stays open · security fixes always land in community · development happens in the open.

Community	Enterprise
All 16 policy types	SSO: OIDC and SAML 2.0
PKI, HSM, Kerberos, LDAP / AD	SCIM 2.0 provisioning
MFA, RBAC, audit, metrics	Node-group-scoped roles
LUKS recovery-key escrow	Compliance reports with history and signed PDFs
Export / import, deploy wizard	Support from the people who build it

What comes next (and where help is welcome)

v0.10.0 and v0.10.1, released this week

- ✓ Five new types: Services & timers, SELinux, AppArmor, Software updates, Disk encryption
- ✓ Restart-safe server state, editions and extension points
- ✓ v0.10.1: disk encryption on a default Fedora install, policy stream survives a suspend, signed checksums

After that

- ○ Agent auto-update
- ○ Git sync for policies, signed bundles
- ○ Console translations, Bulgarian first
- ○ End-to-end tests in CI, more than one active server replica
- ○ Policies for many more desktop applications

Candidate policy types. Which one first?

- ? CA trust store and machine certificates
- ? NetworkManager: Wi-Fi and VPN profiles
- ? CUPS printers
- ? sudoers · SSH client and server
- ? USBGuard · auditd rules
- ? LibreOffice

> Which one first? Tell us at the stand: the most requested type goes into the v0.11 plan. Proposals and pull requests are welcome for everything on this slide.

Demo: from the console to the desktop in seconds

1. The console: fleet, groups, bindings with priorities.
2. A new **Firefox** policy: homepage, locked. Draft, release, bind to the group.
3. The laptop gets it within seconds: desktop notification, Firefox shows it.
4. **Tampering**: edit `policies.json` by hand. It comes back at once; the tamper event is in the audit log.
5. Compliance and audit: what the administrator sees. Time permitting: `setenforce 0` reverted, Session Access, or a package install.

> Remember the hand you raised? This is the same task you were doing over SSH.

Bor

Get involved

getbor.dev

github.com/VuteTech/Bor

demo.getbor.dev · demo / demodemodemo

getbor.dev/publications/openfest2026 · these slides

The Bor stand

Both days. Bor running live: release a policy and watch it land on a desktop.

We are looking for

Pilot organisations: schools, municipalities, companies, hackerspaces. Distro testers. Translators.

Contribute

Issues, pull requests, new policy types, documentation. A star on GitHub helps too.

AGPL-3.0 · © 2026 Vute Tech LTD and contributors · blago.petrov@vute.tech · bor@vute.tech

Questions?

Blagovest Petrov · Founder, Vute.Tech · blago.petrov@vute.tech · getbor.dev

Take the demo policies home: dev.vute.tech/Conferences/OpenFest2026-Bor-Policies

War stories about managing Linux machines are welcome too.